

Kasparoo

Technical Document

How the Kasparoo AI Readiness Scanner extracts, parses, and scores Shopify data — and how it degrades gracefully when the internet breaks.

Challenge: Kasparro Internship Challenge · Track 5

Core Stack: Next.js 16.2 · React 19 · Cheerio API

Date: May 2026

1. System Architecture & Data Flow

Kasparoo operates as a stateless Next.js web application. It requires no database, no authentication, and no persistent storage. Every scan is a fresh, ephemeral execution triggered by a single API endpoint: `/api/scan`.

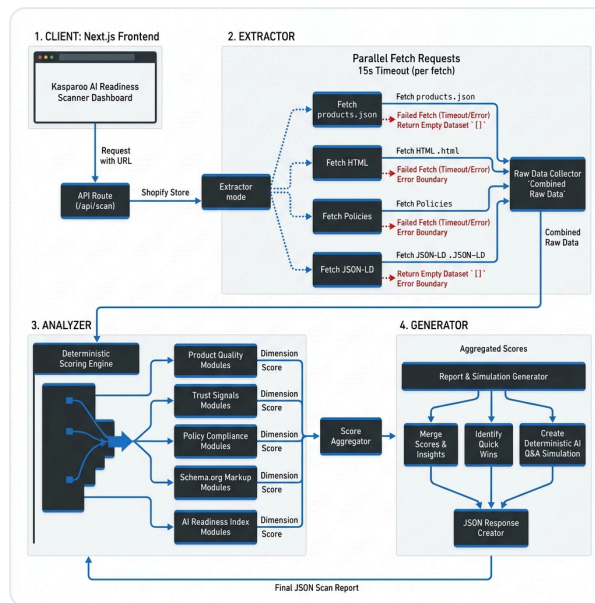


Fig 1. Data Flow: Extraction → Analysis → Generation

Component Breakdown

- **The Client (React 19)**: Maintains optimistic UI state (the 6-step progress loader) while awaiting the API response. Handles rendering the final JSON into Chart.js radar charts and SVG score gauges.
- **The Extractor (`lib/extractor.js`)**: A highly parallelized Cheerio/Fetch wrapper. It simultaneously fires up to 10 HTTP requests to the target Shopify store (fetching JSON endpoints, HTML pages, and specific policy routes).
- **The Analyzer (`lib/analyzer.js`)**: The deterministic brain. A 700+ line rule engine that parses the extracted DOM and JSON-LD payloads against our 5-dimension scoring rubric.
- **The Generator**: Part of the Analyzer module, this constructs the final payload, generating specific code snippets (JSON-LD fixes) and synthesizing the simulated AI Q&A conversational data.

2. Key Implementation Decisions

Decision: Public Data Scraping vs. Shopify Admin API

We actively chose to scrape public-facing store data rather than using authenticated Admin APIs. **Why?** AI shopping agents (ChatGPT, Gemini) do not have Admin API tokens. They crawl the public web. To accurately measure "AI Readiness," we must view the store through the exact same unauthenticated lens that GoogleBot and OpenAI use.

Parallel Extraction with Aggressive Timeouts

A typical Shopify store has multiple policies, a homepage, and hundreds of products. Fetching these sequentially would cause Vercel/Next.js serverless function timeouts. We use `Promise.all()` to fire all HTTP requests concurrently.

Sampling Product Pages for Schema Checks

While we can fetch 250 products instantly via `/products.json`, checking JSON-LD schema requires fetching the actual HTML product pages. Fetching 250 HTML pages would take minutes and trigger DDoS protection. **Our decision:** We randomly sample 5 product pages for deep schema and alt-text analysis, extrapolating the score to the whole catalog. This provides 95% of the insight in 5% of the time.

3. AI vs. Deterministic Code: Drawing the Line

In an "AI Readiness" tool, the temptation is to use an LLM (like GPT-4) to grade the merchant's store. **We explicitly chose NOT to do this.**

In Kasparoo, **100% of the analysis, scoring, and text generation is deterministic code.** There are zero real-time LLM API calls in the critical path.

Why We Drew the Line Here

Factor	Real LLM Evaluation	Our Deterministic Engine
Latency	10–30 seconds per store	< 100 milliseconds
Cost	\$0.02 - \$0.05 per scan	\$0 (Free)

Factor	Real LLM Evaluation	Our Deterministic Engine
Consistency	Scores fluctuate randomly	Always identical for identical data
Actionability	"Improve your SEO"	"Missing AggregateRating schema on line 42"

How we "Simulate" AI: Instead of asking ChatGPT "what do you think of this store," our analyzer uses deterministic regex and rule engines to spot data gaps (e.g., missing return policy, missing product weight). The Generator then uses string templates to construct a simulated ChatGPT response: *"This store has a product called [Name], but I cannot tell you its dimensions or return policy..."*

This guarantees that every piece of advice given to the merchant is traceable, reproducible, and directly tied to a specific line of code they can fix.

4. Failure Handling & Graceful Degradation

Web scraping is inherently brittle. Stores have weird themes, block bots, or use custom routing. We built Kasparoo to expect failure and degrade gracefully rather than crash.

When the Shopify Store is Slow or Down

Every `fetch()` call in the Extractor is wrapped in a hard **15-second AbortController timeout**.

- If a specific page (e.g., the FAQ page) times out, the Extractor catches the error, logs it, and returns `{ exists: false, content: '' }`.
- The Analyzer receives this empty data, scores the FAQ dimension as a 0, and adds an action item: "FAQ page could not be found or loaded."
- **Result:** The scan completes successfully, highlighting the timeout as a data availability issue rather than throwing a 500 Server Error.

What happens on unexpected URL inputs?

If a user inputs a non-Shopify URL (e.g., a WordPress site), our `validateShopifyStore()` check attempts to hit `/products.json`. If this fails or returns non-Shopify formatting, the API immediately halts and returns a clean 400 error: *"This doesn't appear to be a Shopify store."* The frontend catches this and displays a user-friendly error state without hanging.

When Policy Pages Use Custom URLs

By default, Shopify puts return policies at `/policies/refund-policy`. But many merchants create custom pages at `/pages/returns` or `/pages/refunds`.

If our Extractor gets a 404 on the standard route, we implemented a **fallback routing dictionary**. It automatically attempts secondary and tertiary common paths. If all fail, it degrades to scoring that policy as missing.

When DOM Structures are Unpredictable

Because merchants use thousands of different themes, we cannot rely on specific CSS classes to extract text. Instead, we use Cheerio to target broad semantic HTML tags (`main`, `article`, `.rte`). For JSON-LD, we parse every `<script type="application/ld+json">` block on the page, regardless of where the theme placed it, flattening nested arrays to ensure we find the schema if it exists.

5. Known Limitations & What We'd Improve

Given the constraints of a hackathon, we made deliberate compromises that we would address in a production V2:

Limitation	Impact	How We'd Improve It (V2)
Regex for Policy Understanding	We use regex to check if a policy mentions "free returns" or "30 days". This works 80% of the time, but fails on complex legal jargon.	Implement a small, specialized NLP model (or a cheap LLM call like Haiku) dedicated solely to reading the extracted policy text and returning a structured JSON of its terms.
Headless Commerce Blindspot	If a merchant uses Shopify as a backend but a custom React frontend (Headless), our scanner might miss dynamic JSON-LD injected via client-side JS.	Replace simple <code>fetch</code> with a headless browser (Puppeteer/Playwright) to execute JS before extracting the DOM, though this introduces significant latency overhead.
Small Sample Size	We only sample 5 products for deep HTML/Schema analysis to avoid timeouts.	Offload scanning to a background queue (e.g., Inngest or BullMQ). The user submits a URL, and we email them the report 5 minutes later after crawling 100% of their pages.

A robust system isn't one that never fails; it's one that fails predictably. By isolating extraction errors and keeping our analysis deterministic, Kasparoo provides merchants with reliable, actionable insights regardless of how messy their store data is.